

Analyse de Sentiment sur Textes Financiers

Benchmarking Classical ML, Deep Learning et Transformers

Housseem MAJED Victoria VIDAL

Université Paris 1 Panthéon-Sorbonne
Master MoSEF – Deep Learning / NLP

Avril 2026

Plan

- 1 Introduction
- 2 Dataset
- 3 Modeles developpes
- 4 Implementation
- 5 Resultats
- 6 Dashboard Streamlit
- 7 Difficultes rencontrees
- 8 Conclusion

Problematique

Les marches financiers produisent un volume massif de textes (communiqués de presse, rapports d'analystes, dépêches). Classifier automatiquement le sentiment de ces textes permet d'extraire un signal utile pour l'aide à la décision.

Objectif du projet : comparer trois générations d'approches NLP sur une tâche de classification de sentiment financier en trois classes (positif, neutre, négatif).

- Classical ML (TF-IDF + classifieurs linéaires)
- Deep Learning (BiLSTM)
- Transformers (BERT, DistilBERT, FinBERT)

Source : Malo et al. (2014), JASIST 65(4).

4 846 phrases extraites de communiqués financiers en anglais, annotées par 16 experts selon le point de vue d'un investisseur.

Question d'annotation :

“Cette phrase donne-t-elle une impression positive, négative ou neutre de l'entreprise mentionnée ?”

Seules les phrases avec **accord majoritaire** (>50%) sont incluses.

Classe	Nombre	Part
Neutral	2 879	59.4%
Positive	1 363	28.1%
Negative	604	12.5%

Desequilibre important :

la classe négative ne représente que 12.5% du jeu.

Split stratifie

70% train / 10% validation / 20% test — seed=42, stratifie par classe.

Metrique principale : Macro-F1

- Moyenne equitable du F1 par classe
- La classe *negative* (12.5%) pese autant que *neutral* (59.4%)
- Un modele predisant toujours “neutral” obtiendrait 59% d'accuracy mais un macro-F1 proche de 0

Metrique secondaire : Accuracy — reportee pour reference, mais trompeuse sur donnees desequilibrees.

1. Classical ML — TF-IDF + Classifieurs

Pipeline : texte $\rightarrow$ vecteurs TF-IDF creux (unigrammes + bigrammes, 20K features, TF sublineaire) $\rightarrow$ classifieur.

Logistic Regression

- Regularisation L2, solver lbfgs
- Rapide a entrainer (<1s)

Naive Bayes

- MultinomialNB, alpha=0.1
- Hypothese d'indépendance naive

Avantages : rapide, interpretable, pas de GPU nécessaire.

Limites : ignore l'ordre des mots, le contexte et la sémantique.

2. Deep Learning — Bidirectional LSTM

Architecture : Embedding (64d, 15K vocab) → BiLSTM 2 couches (128 unites) → Dense → Softmax.

- Entrainement : 20 epochs, batch size 32, Adam optimizer
- Early stopping sur la loss de validation
- Entrainement sur CPU et GPU (40s)

Avantages : capture l'ordre des mots et les dependances locales.

Limites : sans pre-entrainement et avec seulement 5K exemples, le modele ne generalise pas sur la classe minoritaire (0% recall sur *negative*).

3. Transformers — BERT, DistilBERT, FinBERT

Principe : modeles pre-entraînés sur de larges corpus, fine-tunes sur notre dataset.

BERT

110M params

Pre-entraîne sur BookCorpus +
Wikipedia

Fine-tuning : 5 epochs, $lr=2e-5$

DistilBERT

66M params

Version distillée de BERT (40%
plus légère)

Même hyperparamètres

FinBERT

110M params

Further pre-entraîne sur corpus
financier

Comprend : *restructuring*,
write-down, *impairment*...

Optimisations GPU : fp16, gradient checkpointing, VRAM cap à 85%, gradient accumulation.

Structure modulaire :

- `src/config.py` : chemins, hyperparametres, registre des modeles
- `src/data.py` : chargement, nettoyage, split stratifie
- `src/metrics.py` : evaluation (accuracy, macro-F1, weighted-F1)
- `src/inference.py` : chargement modele, pipeline HuggingFace, prediction

Scripts d'entrainement :

- `train_baselines.py` : TF-IDF + LogReg/NB
- `train_lstm.py` : BiLSTM via TensorFlow
- `train_transformer.py` : BERT/DistilBERT/FinBERT
- `build_leaderboard.py` : agregation des resultats

Environnement :

Python 3.11, uv, PyTorch 2.11+cu128,
GPU NVIDIA RTX 5070 Laptop (8 Go VRAM)

Benchmark comparatif

Rang	Modele	Accuracy	Macro-F1	Type	Temps
1	FinBERT	88.1%	87.7%	Transformer	82s
2	DistilBERT	86.8%	86.0%	Transformer	51s
3	BERT	85.5%	83.8%	Transformer	111s
4	TF-IDF + NaiveBayes	71.3%	62.7%	Classical	<1s
5	TF-IDF + LogReg	72.4%	60.1%	Classical	<1s
6	BiLSTM	63.7%	37.3%	Deep Learning	41s

Observations clés

- Le pre-entrainement sur corpus financier (FinBERT) donne un avantage net
- DistilBERT surpasse BERT complet (moins d'overfitting avec peu de donnees)
- Les baselines ont une accuracy correcte mais echouent sur la classe minoritaire
- Le BiLSTM ne generalise pas sans pre-entrainement

Interface web construite avec Streamlit, deployee localement.

Analyze

- Prediction avec un modele au choix
- Comparaison multi-modeles cote a cote
- Explicabilite LIME (mots cles)

Batch Analysis

- Upload CSV de headlines
- Classification en masse
- Export des resultats

Live News (Yahoo Finance)

- API yfinance pour les headlines
- Analyse de sentiment en temps reel
- Jauge de sentiment global

Entree vocale

- Web Speech API (Chrome/Edge)
- Injection JS dans le contexte parent
- Transcription live dans le textarea

Integration yfinance

On utilise la librairie `yfinance` pour recuperer les titres d'actualite associes a un ticker boursier (AAPL, TSLA, MSFT...). Chaque headline est ensuite passee dans le pipeline de prediction du modele actif. On calcule un score de sentiment global (proportion positifs – negatifs) pour fournir un indicateur synthetique.

Web Speech API

L'entree vocale utilise la Web Speech API du navigateur. Le defi technique principal etait que Streamlit execute les composants HTML dans des iframes sandboxes, sans acces au micro. La solution retenue injecte un script dans le document parent via `components.html()`, ce qui permet au `SpeechRecognition` de s'executer dans le bon contexte avec les permissions navigateur.

- ➊ **Gestion de la VRAM** — Le fine-tuning de BERT (110M params) sur une RTX 5070 (8 Go) a necessite fp16, gradient checkpointing et un cap memoire a 85%. Sans ces optimisations, les modeles depassaient la memoire GPU.
- ➋ **PyTorch CUDA sur Windows** — L'installation via `pip install torch` donne une version CPU-only. Il a fallu configurer un index PyTorch specifique (`pytorch-cu128`) dans `pyproject.toml`.
- ➌ **Entree vocale dans Streamlit** — Les iframes Streamlit bloquent l'accès micro. Plusieurs approches ont echoue (`st_javascript`, `components.html` directement) avant de trouver la solution d'injection dans le contexte parent.
- ➍ **Synchronisation etat Streamlit/DOM** — Les modifications du textarea par JavaScript ne mettaient pas a jour l'etat interne de Streamlit. Resolution via le setter React natif et les callbacks `on_click`.
- ➎ **Desequilibre des classes** — Le BiLSTM predisait systematiquement "neutral", obtenant 0% recall sur *negative*. Ce comportement a confirme l'importance du pre-entrainement.

Bilan

FinBERT domine le benchmark (87.7% macro-F1) grâce à son pré-entraînement sur corpus financier. L'écart avec les baselines classiques confirme que le transfert de connaissances est décisif sur un petit jeu de données spécialisé.

Perspectives :

- Elargir le dataset (SEC filings, earnings calls)
- Tester des modèles plus récents (DeBERTa, LLMs few-shot)
- Déploiement cloud avec API REST
- Analyse multi-langue (français, allemand)

Merci de votre attention.